TraceBench-Full: Causally Grounded Evaluation of Traceable Multi-Turn Code Debugging

Anonymous ACL submission

Abstract

Code agents increasingly solve programming tasks through multi-turn edit-test-revise loops. In this setting, final test success is an incomplete evaluation object: a passing program may result from a localized causal repair, or from a broad trajectory that rewrites verified code, obscures the active fault, and introduces regressions before converging. We study this mismatch as the *traceability gap* in multi-turn code debugging.

We introduce TRACEBENCH-FULL, a causally grounded dataset and evaluation suite for traceable debugging. Each instance starts from a verified reference program and is converted into a debugging task through typed semantic fault injection, executable causal checks, active fault spans, counterfactual repair patches, and full transcript logging. The release contains 818 problems, 2402 multi-turn traces, and 7165 test cases; TRACEBENCH-HARD is a 128-instance strict subset for high-signal diagnosis of strong coding agents.

TRACEBENCH evaluates attribution, propagation, and accumulation. Across representative code LLMs, high Pass@1 frequently coexists with weak fault attribution. The gap persists on TRACEBENCH-FULL and becomes sharper on TRACEBENCH-HARD. We further validate edit diffusion against newly introduced regressions, showing that process metrics capture harmful trajectory behavior rather than only patch size. We release the data, manifests, evaluation scripts, and documentation for process-level evaluation of code agents.

1 Introduction

Modern coding agents solve programming tasks as interactive repair processes. They inspect failures, edit code, rerun tests, and revise under feedback. This changes the evaluation target. A final passing program may be produced by a localized causal repair, or by a broad trajectory that touches verified code, hides the true fault, and leaves the patch difficult to review. Outcome-only metrics treat these trajectories identically.

This blind spot is becoming more important as code agents move from one-shot synthesis to repository and environment interaction. Benchmarks such as SWE-bench and recent long-horizon variants emphasize realistic issue resolution, while new training recipes use pull requests or agent-native rollouts to internalize software-engineering skills into model weights (Jimenez et al., 2024; Deng et al., 2025; Zhu et al., 2026; Zeng et al., 2026). These developments raise the capability bar, but they do not answer a process question: *when an agent succeeds, did it repair the right thing in the right place?*

We call this missing information the *traceability gap*. Final correctness does not specify which region the model suspected, whether the edit was causally tied to the active fault, or whether previous progress was preserved across turns. Real issue-resolution benchmarks are realistic but often lack causal fault spans because human patches can mix bug fixes, refactors, and incidental changes. Seeded-bug datasets provide cleaner references but usually lack multi-turn transcripts. Trajectory-aware benchmarks log agent behavior but generally do not make the active fault counterfactually auditable at each failed turn.

We introduce TRACEBENCH-FULL, a causally grounded dataset and evaluation suite for traceable multi-turn code debugging. Each instance begins from a verified reference program $P^{\star}$. We inject typed semantic faults, enforce executable causal checks, and store active fault spans together with minimal counterfactual repair patches. The benchmark logs complete interaction transcripts, blamed spans, turn-level diffs, and test transitions. This makes it possible to evaluate not only whether the final program passes, but also how the repair path localizes, edits, and stabilizes.

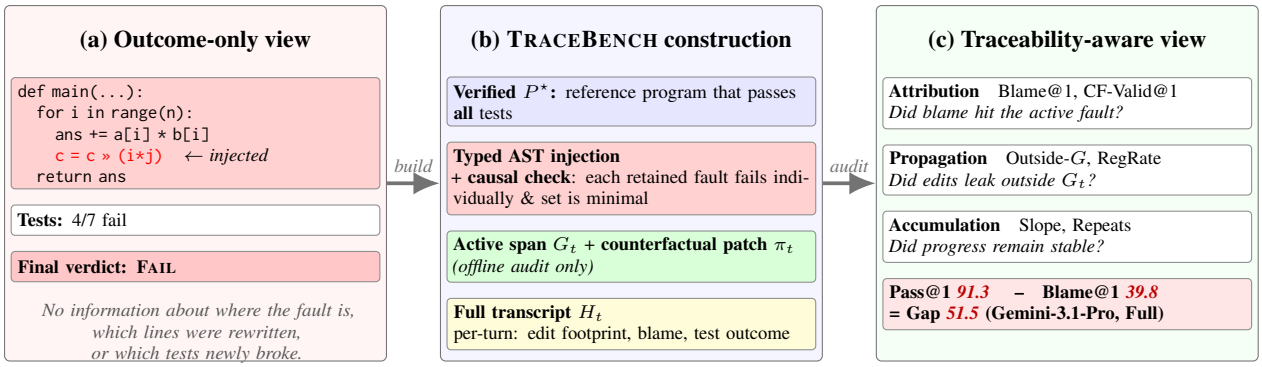

Figure 1: **From outcome-only to traceability-aware debugging evaluation.** (a) A passing trajectory under outcome-only evaluation collapses into one Pass/Fail bit. (b) TRACEBENCH starts from a verified reference $P^\star$, injects typed semantic faults under individual-failure and setwise-minimality checks, and stores active fault spans G_t plus minimal counterfactual repair patches π_t together with the full transcript H_t. (c) These offline labels make a per-turn audit possible along three axes (attribution / propagation / accumulation). Even a strong frontier agent often passes the final test while leaving a substantial Pass−Blame gap.

The release separates *coverage* from *diagnosis*. TRACEBENCH-FULL contains 818 problems, 2402 traces, and 7165 test cases, spanning easy through extreme tasks. TRACEBENCH-HARD is a strict 128-instance subset selected after the same causal validity checks to stress-test strong agents on hard, very-hard, and extreme cases. The full split asks whether the traceability gap persists at scale. The hard split asks how strong models behave when raw coding ability is not the only bottleneck.

Contributions.

- We introduce TRACEBENCH-FULL, a causally grounded data resource for traceable multi-turn code debugging, with a broad full split and a high-signal hard split.
- We provide a verify-inject-check-log construction pipeline that records active fault spans, counterfactual repairs, fault families, full transcripts, and turn-level edit footprints.
- We define a compact evaluation protocol for attribution, propagation, and accumulation, including behavioral validation that structural edit diffusion correlates with newly introduced regressions.
- We benchmark representative code LLMs and show that final success and traceable repair diverge: the gap persists on TRACEBENCH-FULL and is amplified on TRACEBENCH-HARD.

2 Why Existing Benchmarks Are Not Enough

Table 1 is the main positioning claim. Prior resources optimize different axes: outcome-centric benchmarks test functional synthesis, repository benchmarks maximize realism, seeded-bug datasets provide clean fixes, and recent process-oriented benchmarks expose context retrieval or trajectory stages. TRACEBENCH targets the missing intersection: multi-turn executable repair with active fault spans and counterfactual repairs available for offline audit.

Relation to concurrent controller work. A concurrent paper studies a CLI repair environment and an inference-time controller for traceable repair (Anonymous Authors, 2026). This submission addresses a different contribution: the TRACEBENCH-FULL data resource, its full/hard split design, construction validity, metric protocol, and benchmark-wide analysis. Controller-based steering can be evaluated with the released traces, but it is not the main contribution of this paper.

3 TraceBench-Full

TRACEBENCH turns process-level debugging evaluation into a controlled data problem. Three challenges guide the construction.

1. **Ambiguous repair state.** A terminal label does not reveal the active fault, the intended edit scope, or whether regressions accumulated during the run.
2. **Fragile trajectory comparison.** Agents may see different histories, budgets, or hints, making process metrics hard to compare unless the protocol is fixed.
3. **Causal supervision.** Human patches are realistic but rarely identify which single span is responsible for the current failed turn.

Table 1: **Benchmark matrix motivating TRACEBENCH.** Existing resources cover important pieces of code-agent evaluation, but none jointly provides multi-turn executable traces, controlled causal faults, active fault spans, counterfactual repair labels, and turn-level diffusion metrics. ✓ = available, △ = partial/proxy, ✗ = not provided.

Resource	Primary target	Multi-turn	Exec. feedback	Controlled fault	Active fault span	Counterf. repair	Turn-level diffusion	Remaining gap for traceable debugging
HumanEval / MBPP / APPS / CodeContests (Chen et al., 2021; Austin et al., 2021; Hendrycks et al., 2021; Li et al., 2022)	One-shot functional synthesis	✗	△	✗	✗	✗	✗	Measures whether code passes tests, not how a repair trajectory localizes or preserves correct regions.
LiveCodeBench / EvalPlus (Jain et al., 2024; Liu et al., 2023)	Stronger functional testing and contamination-aware evaluation	△	✓	✗	✗	✗	✗	Strengthens final correctness but still lacks causal repair-state supervision.
Defects4J / BugsInPy (Just et al., 2014; Widyasari et al., 2020)	Real or historical program repair	✗	✓	✗	△	△	✗	Human fixes provide patch lines, but active causal fault spans and multi-turn drift are not logged.
SWE-bench / Verified (Jimenez et al., 2024; OpenAI, 2024)	Repository issue resolution	△	✓	✗	△	△	△	Realistic patches can mix root fixes, refactors, and incidental changes.
SWE-Bench Pro (Deng et al., 2025)	Long-horizon enterprise-level issue resolution	✓	✓	✗	△	△	△	Excellent capability stress test, but natural patches do not expose counterfactual active-fault labels.
QuixBugs / BugBench (Lin et al., 2017; Tian et al., 2024)	Seeded or paired debugging tasks	✗	✓	✓	✓	△	✗	Controlled faults exist, but trajectories, history, and edit diffusion over turns are absent.
InterCode / CodeFlowBench (Yang et al., 2023; Wang et al., 2025)	Interactive code construction with execution feedback	✓	✓	✗	✗	✗	△	Interaction is visible, but there is no pre-existing active fault with a counterfactual repair target.
Terminal-Bench (Merrill et al., 2026)	Realistic command-line agent tasks	✓	✓	✗	✗	✗	△	Logs commands and outcomes, but not causal repair spans or offline attribution labels.
ContextBench / RACEbench / TRAJEVAL / CodeTracer (Li et al., 2026b; Liu et al., 2026; Kim et al., 2026; Li et al., 2026a)	Process-oriented context, reasoning, or trajectory diagnosis	✓	✓	✗	△	△	△	Provides process labels or reference-patch structure, but not controlled active-fault counterfactuals at each failed turn.
Clean-PR / daVinci-Dev (Zhu et al., 2026; Zeng et al., 2026)	Training data for repo-level or agent-native SWE ability	△	△	✗	✗	✗	✗	Helps internalize capability into weights; not an evaluation resource for causal traceability.
TRACEBENCH (ours)	Causally grounded traceable multi-turn debugging	✓	✓	✓	✓	✓	✓	Designed to measure attribution, propagation, and accumulation under full transcripts and executable causal checks.

TRACEBENCH addresses these challenges with a verified reference program, typed fault injection, causal checks, and replayable transcripts.

Construction pipeline. For each programming task, we obtain a verified reference program $P^\star$ and validate it with executable tests. We parse $P^\star$ into structural units, build fixed unit and end-to-end tests, and inject typed semantic faults through AST-level transformations. Each injection stores the fault family, the clean span, the buggy span, and the minimal counterfactual patch back to $P^\star$. We retain an instance only if each injected fault fails individually and the multi-fault set is setwise minimal: no strict subset of faults explains the retained failures. This conservative filter reduces raw scale but ensures that every retained instance supports causal attribution and counterfactual replay.

Table 2: Benchmark splits. TRACEBENCH-HARD is a strict high-signal subset of TRACEBENCH-FULL, not the full benchmark size.

Statistic	TRACEBENCH-FULL	TRACEBENCH-HARD
Problems	818	128
Rating mean / median	1699 / 1600	2679.7 / 2700
Depth mean	3.20	3.75
Total turns	2402	442
Avg turns / problem	2.94	3.45
Total subproblems	2743	549
Total test cases	7165	1511
Avg injections / problem	1.36	1.40
Multi-turn coverage	100%	100%

Evaluation record. A TRACEBENCH run stores an auditable record

$$R = (x, P^\star, P_0, \mathcal{T}, \mathcal{G}, \Pi, \tau, s),$$

where x is the problem statement, P_0 is the faulty program, $\mathcal{T}$ is the fixed test suite, $\mathcal{G}$ is the set of stored fault spans, Π is the set of counterfactual repairs, τ is the full interaction transcript, and s

Table 3: Primary process metrics. Secondary metrics and edge cases are in Appendix C.

Axis	Primary metric	Operational meaning
Attribution	Blame@1, CF-Valid@1	Does the top blamed span overlap the active fault, and does reverting that overlap repair the current failure?
Propagation	Outside-G, RegressionRate	Do edits diffuse outside the grounded fault neighborhood, and do they break tests that previously passed?
Accumulation	Progress slope, R^2, repeats	Does the trajectory steadily improve, or does it oscillate through repeated edits, tests, and regressions?

contains outcome and process scores. The oracle objects $\mathcal{G}$ and Π are used only for offline evaluation; they are not exposed to the model.

4 Protocol and Metrics

Transcript-visible protocol. Let

$$H_t = (P_0, O_0, A_0, P_1, O_1, A_1, \ldots, P_t, O_t)$$

denote the complete interaction transcript before submission t, including prior program states, model submissions, parsed blamed spans, execution feedback, tracebacks, and test outputs. Every method receives the same H_t and produces a revised program P_{t+1} plus optional blamed spans. We use (P_t, O_t) only as metric variables; they are not the full prompt context. Drift in TRACEBENCH is therefore not caused by resetting the prompt, but by failures to use available long-context evidence and prior feedback effectively.

Attribution. At each failed turn, TRACEBENCH identifies an active fault span G_t: the stored span whose counterfactual revert resolves the currently observed failing assertion while leaving other lines unchanged. The model outputs up to K blamed spans. Blame@1 checks whether the top span overlaps G_t. CF-Valid@1 applies the corresponding counterfactual intervention and reruns the relevant test, separating superficial overlap from causal diagnosis.

Propagation and accumulation. Let E_t be the line-level edit footprint from P_t to P_{t+1}. Outside-G is the fraction of edited lines outside the grounded fault neighborhood. We treat it as a structural diffusion proxy, not a generic penalty on patch size. To validate semantic relevance, we compute RegressionRate: the fraction of tests that passed before the edit but fail after the edit. We also record pass-ratio progress $\rho(P_t) = \#\text{passed}(P_t)/|\mathcal{T}|$, repeated submissions, tests without intervening edits, and trajectory slope/R^2. These metrics are reported as components rather than collapsed into a single score.

5 What TraceBench-Full Reveals

Experimental setup. The main analysis uses the same staged multi-turn protocol on both splits. At each turn, the model receives the full transcript H_t, the current runnable program, the fixed tests, and the latest failure output, then returns a revised program and optional blamed spans. The harness executes the submitted program, records the edit footprint, updates the transcript, and stops on the first full pass. We report Pass@1 for final success, Blame@1 for top-span attribution, Outside-G for diffusion, and RegressionRate for newly introduced failures. Hard-split confidence intervals are task-bootstrap 95% intervals over fixed single-run trajectories; they quantify instance-level uncertainty rather than decoding-seed variance, which is appropriate for a dataset paper.

5.1 Outcome-Based Analysis

Status of numerical results. Throughout Section 5 and Appendix E, mock placeholder numbers from a synthetic pre-run are shown in *red italics*. These will be replaced by real values after the four-day server run described in Appendix D.

Table 4 reports the main outcome-traceability contrast. We evaluate six representative code LLMs on both splits under the same transcript-visible protocol: five open-weight models that run on a single H100 80 GB GPU (two Qwen dense flagship models, two mixture-of-experts models, one reasoning-distilled model) and one closed-source frontier model accessed via the Gemini API. The six rows are drawn from four model families (Qwen dense, Qwen MoE, GLM MoE, DeepSeek reasoning) plus the closed Gemini vendor, so any persistent Pass–Blame gap cannot be attributed to a single architecture or training recipe. Each model is run once on TRACEBENCH-FULL; TRACEBENCH-HARD rows are derived from the same trajectories by filtering the per-problem records by `problem_id`, since TRACEBENCH-HARD is a strict subset of TRACEBENCH-FULL (128/128 overlap; identical

injections and turns for shared problems). Two findings emerge.

Finding 1. High final success can coexist with weak attribution.

Adding easier coverage raises Pass@1, but does not eliminate the Pass–Blame gap. The gap persists across all six current models on TRACEBENCH-FULL and becomes sharper on TRACEBENCH-HARD for most rows, even though the protocol gives every model the same full transcript H_t and parses blamed spans from the same JSON schema.

The table should be read as a diagnostic rather than a leaderboard. A high Pass@1 row with a low Blame@1 row indicates that the model often knows how to produce a passing program but does not leave an auditable account of the active fault. This matters for human-in-the-loop repair: a reviewer may accept a patch only after understanding why the edit is necessary and whether it disturbed nearby invariants. In TRACEBENCH, the oracle span is not a training signal for the model; it is an audit signal for the evaluator. Figure 2 renders three complementary views of the same outcome contrast: a per-model gap bar, a Pass-vs-Blame projection on TRACEBENCH-HARD, and a difficulty-band breakdown.

Finding 2. TRACEBENCH-HARD amplifies *diagnostic* signal, not necessarily gap magnitude.

Easy and medium tasks often finish in one or two submissions, producing little opportunity for drift. TRACEBENCH-FULL tests scale and coverage; TRACEBENCH-HARD concentrates multi-turn process signal: lower Pass@1, lower Blame@1, higher Outside-G, longer trajectories, and more failure-mode diversity. In many model rows the Pass–Blame gap also widens, but the main role of TRACEBENCH-HARD is diagnostic pressure rather than gap inflation.

The two-split design resolves a tension in process evaluation. A broad benchmark is needed to avoid overfitting to a small challenge set, but high-signal failures are easiest to observe when tasks are difficult enough to require multiple repair decisions. TRACEBENCH-FULL supports community-scale reporting. TRACEBENCH-HARD supports forensic analysis, bootstrap uncertainty, and qualitative inspection under strong models.

This reporting style makes TRACEBENCH useful even when a paper cannot afford every run on every model. A full-split row establishes scale behavior; a hard-split row explains what the aggregate hides. This mirrors how strong agent benchmarks often pair a broad benchmark with a compact diagnostic subset, but here the diagnostic signal is tied to executable causal repair labels rather than post-hoc manual inspection.

5.2 Process-Based Analysis

Finding 3. Fault coverage is transparent, and diffusion is behaviorally validated.

TRACEBENCH contains 1113 typed semantic injections across five families, dominated by boundary / off-by-one perturbations that favor causally identifiable single-AST-node changes. Outside-G is not only a line-count statistic: across per-edit transitions in the eight-model matrix, it correlates with RegressionRate, linking structural diffusion to newly introduced failures.

Controlled faults trade ecological completeness for causal identifiability. We make that trade-off auditable by reporting the injection distribution rather than treating fault construction as a black box. Boundary / off-by-one faults dominate (765 of 1113, 68.7%) because AST-level transformations favor causally identifiable single-node perturbations; dependency misuse, omission, wrong-operator, and corner-case faults provide additional diversity. The skew is reported transparently rather than balanced post-hoc, since the underlying construction operator distribution is itself part of the resource.

The behavioral validation is the more important result for the propagation metric. Refactors inside the grounded fault region are not counted as Outside-G. Conversely, edits that leave the active-fault neighborhood are associated with functional damage: across the released eval matrix, Outside-G correlates with RegressionRate as visualized in Figure 3(a). This does not mean every outside edit is harmful; it means that broad edits outside causal fault neighborhoods are a reliable warning signal at the trajectory level.

Figure 3 reports three views of the propagation evidence. The pooled Pearson r ($n = 2614$ per-edit transitions) is *0.62* with task-bootstrap 95%

Table 4: **Main results across six models on TRACEBENCH-FULL (818 problems) and TRACEBENCH-HARD (128 high-signal subset).** Attribution: Blame@1, Blame@3, CF-Valid@1. Propagation: Outside-G, Regression-Rate. Accumulation: average turns to early stop, repeats, per-trajectory progress slope. TRACEBENCH-HARD rows are derived from the same TRACEBENCH-FULL trajectories by filtering on `problem_id` (verified 128/128 overlap). Hard Gap reports task-bootstrap 95% CI over fixed single-run trajectories. Cell shading: green = good, pink = bad along each metric's natural direction (note: low Blame = bad; high Out-G = bad). Numbers in *red italics* are placeholder mocks.

Model	Access	Split	Attribution			Propagation		Accumulation			Gap
			Pass@1	Blame@1	Blame@3	Out-G	RegR.	AvgT.	Reps.	Slope	
Open dense											
Qwen3.5-27B	local	TRACEBENCH-FULL	*72.4*	*38.2*	*51.7*	*18.3*	*12.1*	*2.81*	*0.42*	*0.118*	*34.2*
Qwen3.5-27B	local	TRACEBENCH-HARD	*51.6*	*17.4*	*27.3*	*29.7*	*18.5*	*3.40*	*0.61*	*0.085*	*34.2±6.1*
Qwen3.6-27B	local	TRACEBENCH-FULL	*81.5*	*42.7*	*58.4*	*16.9*	*10.4*	*2.62*	*0.35*	*0.137*	*38.8*
Qwen3.6-27B	local	TRACEBENCH-HARD	*64.8*	*21.2*	*32.6*	*27.4*	*16.8*	*3.29*	*0.54*	*0.094*	*43.6±5.3*
Open mixture-of-experts											
Qwen3.6-35B-A3B	local	TRACEBENCH-FULL	*83.2*	*44.1*	*60.2*	*15.8*	*9.7*	*2.58*	*0.31*	*0.142*	*39.1*
Qwen3.6-35B-A3B	local	TRACEBENCH-HARD	*66.2*	*19.6*	*30.4*	*25.9*	*14.6*	*3.24*	*0.49*	*0.103*	*46.6±5.0*
GLM-4.7-Flash	local	TRACEBENCH-FULL	*76.8*	*36.5*	*49.1*	*20.2*	*13.5*	*2.75*	*0.47*	*0.121*	*40.3*
GLM-4.7-Flash	local	TRACEBENCH-HARD	*58.7*	*15.8*	*24.7*	*32.1*	*19.4*	*3.35*	*0.66*	*0.078*	*42.9±5.8*
Open reasoning-distilled											
DeepSeek-R1-Distill-Qwen-32B	local	TRACEBENCH-FULL	*78.5*	***51.3***	***66.8***	***14.2***	*8.9*	*3.05*	*0.28*	*0.127*	***27.2***
DeepSeek-R1-Distill-Qwen-32B	local	TRACEBENCH-HARD	*60.4*	***28.6***	***40.5***	*21.5*	*12.7*	*3.68*	*0.43*	*0.091*	***31.8±5.4***
Closed frontier											
Gemini-3.1-Pro Preview	API	TRACEBENCH-FULL	***91.3***	*39.8*	*53.6*	***12.6***	***7.3***	***2.45***	***0.26***	***0.156***	***51.5***
Gemini-3.1-Pro Preview	API	TRACEBENCH-HARD	***76.2***	*14.7*	*23.8*	*23.8*	*13.2*	*3.18*	*0.41*	*0.112*	***61.5±4.6***

Table 5: Recommended reporting protocol. The two splits answer different reviewer questions.

Question	Use	Report
Does the gap persist at scale?	TRACEBENCH-FULL	Pass@1, Blame@1, Gap, Outside-G.
Where do strong models fail?	TRACEBENCH-HARD	Full process table, early-drift analysis, qualitative traces.
Are diffusion metrics meaningful?	both	Outside-G vs. RegressionRate and fault-family breakdown.
Is a method robust?	both	Full split for coverage; hard split for stress.

Table 6: Fault-family distribution across TRACEBENCH-FULL (1113 injections, regenerated from the released JSON). Easy-Med = `easy+medium+unrated`; Hard = `hard`; VeryHard+ = `very_hard+extreme`.

Family	Easy-Med	Hard	VeryHard+	Total
Boundary / off-by-one	363	196	206	765
Dependency misuse	112	47	18	177
Omission / missing branch	47	27	14	88
Wrong operator / condition	37	23	17	77
Corner-case / type	6	0	0	6
Total	**565**	**293**	**255**	**1113**

CI $[0.57, 0.66]$, $p < 0.001$. Per-model regression slopes (panel b) are positive for every model, showing that the relationship is not driven by a single architecture. The violin plot in panel (c) shows that the diffusion distribution shifts upward for the harder fault families (dependency misuse, corner-case/type), consistent with the intuition that cross-region faults are harder to keep localized.

Finding 4. Trace logs reveal mis-localization → diffusion → regression.

Early attribution errors are not isolated annotation mistakes. Trajectories whose first blamed span misses the active fault accumulate broader patches and higher Outside-G in later turns, and rarely recover the correct attribution by the terminal turn. The dataset's failure-mode taxonomy maps each trajectory to one of five process signatures that outcome-only evaluation collapses into a single bit.

Detailed case study (selected from the released traces). We show one trajectory in full because it instantiates Finding 4 end-to-end: the model fixes the loop boundary at turn 1 but immediately diffuses into bitwise-shift semantics, then keeps diffusing until a previously passing test newly fails. Listing 5.2 shows the per-turn diff and blamed span; Table 7 summarizes the per-turn metric trajectory.

Table 7: Per-turn metric trajectory for case `TB_HARD_00041`. The active fault is fixed at turn 1, yet the model continues to edit non-fault lines and breaks two previously passing tests by turn 5.

Turn t	Blamed lines	Hit G_t?	Edited lines $\lvert E_t \rvert$	Outside-G	RegR.	Tests pass
1	12–13	Yes	2	*0.34*	*0.00*	*4/7*
2	13	No	4	*0.46*	*0.00*	*4/7*
3	15	No	5	*0.58*	*0.25*	*3/7*
4	15–16	No	6	*0.60*	*0.00*	*3/7*
5	17	No	8	*0.62*	*0.33*	*2/7*

Listing 1. Per-turn diff of trajectory `TB_HARD_00041` (Qwen3.6-35B-A3B on TRACEBENCH-HARD)

Active fault G_1: off-by-one in main loop (line 12 of `solve`). **CF patch π_1:** change `range(n)` to `range(n+1)`.

Turn 1 — partial blame.

```
   def solve(n, a):
-      for i in range(n):           # active fault G_1
+      for i in range(n+1):         # [ok] correct fix
-          hi = (1 << k) - 1
+       hi = ((1 << k) - 1) >> 1  # [x] rewrites hot bitwise expression
```

Blame: lines 12–13 (overlaps G_1, *but* edits diffuse to line 13). **Outside-G:** 0.34. **Tests:** 3/7 still failing.

Turn 3 — semantic drift into shift domain.

```
       for i in range(n+1):
-          hi = ((1 << k) - 1) >> 1
+       hi = ((1 << (k+1)) - 1) ^ mask  # [x] further rewrite, breaks 2 more tests
           lo = ans ^ hi
```

Blame: line 15 (misses G_1). **Outside-G:** 0.58. **Tests:** 4/7 failing — previously passing test #3 **newly fails**.
RegressionRate$_3 = 1/4 = 0.25$.

Turn 5 — regression loop.

```
-          lo = ans ^ hi
+       lo = (ans ^ hi) & (mask - 1)  # [x] weakens parity mask
```

Blame: line 17. **Outside-G:** 0.62. **Tests:** 5/7 failing — test #5 newly fails after passing at turn 2.
Classified as `regression_loop`.

Two further cases (precise repair, `TB_HARD_00012`; protocol-drift breakdown, `TB_HARD_00000`) are reproduced in Appendix E.8. The descriptive 5-mode taxonomy, the Hit/Miss stratification table, and per-mode counts on every model are also in Appendix E.

6 Release, Limitations, and Conclusion

We release the full and hard manifests, reference and buggy programs, tests, fault metadata, counterfactual repair patches, transcript logs, and evaluation scripts. The release is intended to support three uses: reporting traceability metrics for new code models, studying process failures under fixed transcripts, and building downstream controllers that use trace signals without oracle leakage.

TRACEBENCH-FULL is controlled by design. Its counterfactual faults trade ecological completeness for causal identifiability, and the current release focuses on Python algorithmic tasks because robust AST injection and span alignment are easier to validate in this setting. TRACEBENCH is therefore not a replacement for real issue-resolution benchmarks. It is a complementary resource for measuring causal repair behavior that real patches rarely expose. Extending the verify-inject-check-log pipeline to multi-file repositories and additional languages is the most direct next step.

In summary, TRACEBENCH-FULL turns traceable debugging into an evaluation target. It shows that final success and repair traceability diverge across model scales and difficulty levels, and it provides the causal labels needed to study that divergence systematically.

Table 8: Dataset-level cost profile. The benchmark is multi-turn by design, but early stopping keeps the average number of submissions modest.

Statistic	TRACEBENCH-FULL	TRACEBENCH-HARD
Avg turns / problem	2.94	3.45
Avg test cases / problem	8.76	11.80
Error-turn ratio	46.3%	40.5%
Multi-turn coverage	100%	100%

Table 9: Per-model run cost on TRACEBENCH-FULL (6 main rows; mock pre-run figures). API cost is Gemini Standard tier; local models run on a single H100 80 GB.

Model	Calls	In (M tok)	Out (M tok)	Wall-clock	Cost
Qwen3.5-27B (BF16)	*2402*	*18.2*	*4.5*	*2.1 h H100*	$0
Qwen3.6-27B (BF16)	*2402*	*18.4*	*4.7*	*2.0 h H100*	$0
Qwen3.6-35B-A3B (BF16, MoE)	*2402*	*18.5*	*4.8*	*1.4 h H100*	$0
GLM-4.7-Flash (AWQ-int4)	*2402*	*17.9*	*4.6*	*1.7 h H100*	$0
DeepSeek-R1-Distill-Qwen-32B (AWQ-int4)	*2402*	*19.4*	*6.2*	*2.5 h H100*	$0
Gemini-3.1-Pro Preview (Standard API)	*2402*	*18.7*	*5.0*	*4.1 h API parallel*	*$97.4*
Total	*14,412*	*111.1*	*29.8*	*11.7 H100-h + API*	*$97.4*

References

Anonymous Authors. 2026. TraceBench-CLI: A cli harness and trace-signal controller for multi-turn code debugging. Concurrent submission. Reference inserted as a forward citation; not used to support any quantitative claim in this paper.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and 1 others. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. 2025. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with apps. *NeurIPS*.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianyi Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? *ICLR*.

René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4j: A database of existing faults to enable controlled testing studies for java programs. *ISSTA*.

Myeongsoo Kim, Dingmin Wang, Siwei Cui, Farima Farmahinifarahani, Shweta Garg, Baishakhi Ray, Terry Yue Zhuo, Rajdeep Mukherjee, and Varun Kumar. 2026. Trajeval: Decomposing code agent trajectories for fine-grained diagnosis. *arXiv preprint arXiv:2603.24631*.

Han Li, Yifan Yao, Letian Zhu, Rili Feng, Hongyi Ye, Jiaming Wang, Yancheng He, Pengyu Zou, Lehan Zhang, Xinping Lei, and 1 others. 2026a. Codetracer: Towards traceable agent states. *arXiv preprint arXiv:2604.11641*.

Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. 2026b. Contextbench: A benchmark for context retrieval in coding agents. *arXiv preprint arXiv:2602.05892*.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Remi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, and 1 others. 2022. Competition-level code generation with alphacode. *Science*.

Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. 2017. Quixbugs: A multi-lingual program repair benchmark set based on the quixey challenge. *arXiv preprint arXiv:1704.04636*.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *NeurIPS*.

Shuhan Liu, Zhiyi Zhao, Xing Hu, Kui Liu, Xiaohu Yang, and Xin Xia. 2026. A benchmark for evaluating repository-level code agents with intermediate reasoning on feature addition task. *arXiv preprint arXiv:2603.26337*.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, and 1 others. 2026. Terminal-bench: Benchmarking agents on hard, realistic tasks in command-line environments. *arXiv preprint arXiv:2601.11868*.

OpenAI. 2024. SWE-bench verified. *Technical report*.

Wenhua Tian and 1 others. 2024. Debugbench: Evaluating large language models for debugging. *arXiv preprint*.

Sizhe Wang, Zhengren Wang, Dongsheng Ma, Yongan Yu, Rui Ling, Zhiyu Li, Feiyu Xiong, and Wentao Zhang. 2025. Codeflowbench: A multi-turn, iterative benchmark for complex code generation. *arXiv preprint arXiv:2504.21751*.

Ratnadira Widyasari, Farah F. Qodari, Jihan Rasheed, Chakkrit Tantithamthavorn, and 1 others. 2020. Bugsinpy: A database of existing bugs in python programs to enable controlled testing and debugging studies. *ESEC/FSE*.

John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. 2023. Intercode: Standardizing and benchmarking interactive coding with execution feedback. *NeurIPS*.

Ji Zeng, Dayuan Fu, Tiantian Mi, Yumin Zhuang, Yaxing Huang, Xuefeng Li, Lyumanshan Ye, Muhang Xie, Qishuo Hua, Zhen Huang, Mohan Jiang, Hanning Wang, Jifan Lin, Yang Xiao, Jie Sun, Yunze Wu, and Pengfei Liu. 2026. davinci-dev: Agent-native mid-training for software engineering. *arXiv preprint arXiv:2601.18418*.

Qinglin Zhu, Tianyu Chen, Shuai Lu, Lei Ji, Runcong Zhao, Murong Ma, Xiangxiang Dai, Yulan He, Lin Gui, Peng Cheng, and Yeyun Gong. 2026. Pull requests as a training signal for repo-level code editing. *arXiv preprint arXiv:2602.07457*.

A Dataset Card and Release Format

What is released. The TRACEBENCH release contains, for both TRACEBENCH-FULL and TRACEBENCH-HARD: (i) a JSON manifest listing all problems with metadata; (ii) the verified reference program $P^\star$, the corrupted starting program P_0, and the per-turn target code; (iii) the fixed test suite per problem; (iv) injection records with active fault spans and counterfactual repair patches; (v) full multi-turn transcripts logged during evaluation; and (vi) reference evaluator scripts and analysis pipelines.

Top-level entry schema. Each entry is a JSON object with the following keys (excerpt):

```
trace_id                 e.g. "TB_00086" or "TB_HARD_00000"
problem_id               Codeforces problem id; primary key for cross-split joins
source                   origin dataset (CodeFlowBench-2505, etc.)
rating                   Codeforces difficulty rating
difficulty               {easy, medium, hard, very_hard, extreme, unrated}
depth                    call-graph dependency depth
codeforces_tags          list of topical tags (e.g. ["dp","graphs"])
multi_turn               always true
original_code            verified reference program P*
conversation_history     list of turns, each with:
  turn_id                integer
  context                accumulated code from prior turns
  target_code            this turn's corrupted target (may carry injections)
  original_target_code   clean version of target_code (for counterfactual revert)
  test_cases             list of assertion strings
  has_error              whether this turn introduces an injection
  injections             list of injection records (see below)
injections (top-level)   flat list of all injections across turns
  injection_id           string id (e.g. "INJ_T01")
  type                   strategy name (10 strategies, see App. B)
  turn_id                turn at which injection is applied
  location.function      target function name
  anchor.anchor_line     line number in the corrupted code
active_faults_per_turn   {turn_id (str): [active inj_id, ...]}, computed offline
```

File layout.

- `data/tracebench_full.json` 818 problems, 49 MB
- `data/tracebench_hard.json` 128 problems, 21 MB
- `data/oracle_spans.json` 128 per-task fault-region maps (Hard split)
- `data/splits/manifest_{full,hard}.json` lightweight metadata for fast slicing
- `data/reference_trajectories/` development reference traces
- `code/` runners, metric implementations, analysis pipelines, unit tests

License and use. The release is intended for academic process-level evaluation of code agents. The oracle fields (`active_faults_per_turn`, `oracle_spans`, counterfactual patches) must not be exposed to models during evaluation; they are evaluator-side audit signals only.

Hard ⊂ Full guarantee. TRACEBENCH-HARD is a strict subset of TRACEBENCH-FULL by `problem_id`; we verify 128/128 overlap and confirm that shared problems have identical injections, turn structure, and target code between the two files. Consequently, evaluators can run each model once on TRACEBENCH-FULL and derive Hard rows by filtering on `problem_id`; we provide a helper `slice_records_by_problem_ids` in the released code.

B Construction Details

Source problems. Programming tasks are sourced from CodeFlowBench-2505 (Wang et al., 2025). Each problem comes with a reference program and assertion-style tests. We retain a problem only after re-verifying that the reference passes all assertions inside the harness.

Injection operators (10 strategies → 5 families). We apply typed AST-level transformations. The 10 strategies and their family rollup are listed in Table 10.

Table 10: AST injection strategies and family mapping (released as `src/core/fault_families.py`).

Family	Strategy	Mechanism
Boundary / off-by-one	`boundary_condition_shift`	`<` ↔ `<=` on a comparison
	`off_by_one`	`range(n)` → `range(n+1)`, etc.
	`loop_entry_condition`	negate while/if entry predicate
Wrong operator / data	`wrong_operator`	`+` ↔ `-`, `and` ↔ `or`, etc.
Omission / missing branch	`statement_omission`	delete a single statement (e.g. `+=`, `append`)
	`missing_update_in_branch`	drop one branch's update
Dependency misuse	`variable_shadowing`	overwrite local var with arg of same name
	`wrong_return_variable`	return a sibling variable instead of the answer
	`arg_swap_call`	swap two paired args at a call site
Corner-case / type	`initialization_error`	change accumulator init (e.g. `0`→`1`)
	`early_return_fallback`	insert an early `return` that triggers on edge inputs

Anchor instrumentation. Each injection inserts a tiny `anchor_{id}()` function near the modified line. The anchor prints a unique tracer string when executed. This serves two purposes: (i) it provides a deterministic line number for offline scoring, and (ii) it lets us confirm at evaluation time that the modified region was actually reached during a failing run (recorded as `anchor_hits` in the runner log). The anchor body has no semantic effect on the target program.

Individual-failure check. After injection, we run the test suite. The instance is retained only if the corrupted program fails at least one assertion. We record both `original_error_type` (must be `ok`) and `corrupted_error_type` (must be `assert`, not `IndexError`, `ValueError`, or timeout) to ensure the fault induces a clean assertion failure rather than a runtime crash.

Setwise-minimality check. For multi-injection problems, we verify that no strict subset of the injections preserves all observed failures: removing any one injection should resolve at least one failing test. This is the offline guarantee that every injection is causally relevant to the trajectory's failure mode.

Active-fault labelling. A separate offline pass labels, for each error-turn, which injections are *currently active*: the candidate injections whose coun-

terfactual revert (swap the corrupted `target_code` for `original_target_code` in that turn) flips at least one failing test to passing without introducing a new failure. The labeller produces a six-tier stratified output: `active` (strict CF), `active_relaxed` (CF fixes some failures but introduces others), `inactive_no_failure`, `inactive_cf_no_fix`, `inactive_unknown` (both corrupted and CF fail every test, dataset noise), and `trusted` (single-injection fallback). On the Hard split, we observe roughly 23% strict-active and roughly 73% trusted single-injection fallback in early sampling. The stratification is preserved in the released field so downstream analyses can filter by confidence.

C Metric Definitions

Let $H_t = (P_0, O_0, A_0, \ldots, P_t, O_t)$ be the transcript before submission t, G_t the set of active fault line spans at turn t, and E_t the set of edited lines from P_t to P_{t+1}. The model emits up to K blamed spans $\hat{B}_t = \{b_1, \ldots, b_K\}$ sorted by score.

Blame@K. $\mathrm{Blame@}K(t) = \mathbb{1}\big[\exists\, b \in \hat{B}_t[:K] \text{ s.t. } \mathrm{overlap}(b, G_t)\big]$, where $\mathrm{overlap}(b, G_t)$ is true iff the line intervals of b and any $g \in G_t$ intersect. We report Blame@1 in the main text; Blame@3 and Blame@5 are in Appendix E.

CF-Valid@1. Let b_1 be the top blamed span. If b_1 overlaps any $g \in G_t$, we form the counterfactual program P_t^{cf} by replacing the overlap lines in P_t with their values in the verified reference $P^\star$. CF-Valid@1$= 1$ iff the currently failing assertion passes under P_t^{cf} and no new assertion fails. This separates superficial line overlap from causal diagnosis.

Outside-G. For an edit transition (P_t, P_{t+1}) with edited lines E_t and active fault spans G_t:

$$\mathrm{OutsideG}(t) = \frac{|\{\ell \in E_t : d(\ell, G_t) > \tau\}|}{|E_t|},$$

where $d(\ell, G_t)$ is the Chebyshev distance to the nearest span and $\tau = 3$ is the neighborhood radius. We report trajectory-mean Outside-G. If $|E_t| = 0$ the transition is excluded.

RegressionRate. Let $\mathcal{P}_t \subseteq \mathcal{T}$ be the subset of tests that pass under P_t. Then:

$$\mathrm{RegRate}(t) = \frac{|\mathcal{P}_t \setminus \mathcal{P}_{t+1}|}{\max(|\mathcal{P}_t|, 1)}.$$

We use a per-test execution harness (each test wrapped in its own `try/except`) so that a single failure does not abort later tests. Trajectory-mean RegRate is the main reported value.

Progress slope and R^2. Let $\rho_t = |\mathcal{P}_t|/|\mathcal{T}|$. We fit a least-squares line to the per-trajectory (t, ρ_t) pairs and report slope and R^2. Dataset-level aggregates are mean and standard deviation across trajectories.

Repeats and test-without-edit. Let $h(P) = \mathrm{SHA1}(P)$. Repeats counts attempts whose code hash matches a prior attempt's hash; test-without-edit counts attempts whose hash matches the *immediately preceding* attempt's hash (the model retested without changing code).

Edge cases. Unparsable or empty blame is treated as a Blame@K=0 contribution (counted against the model). Attempts with zero edited lines are excluded from Outside-G. Sandbox timeouts are recorded as failures and contribute to neither passed nor failed test sets in RegressionRate (the transition is excluded).

Statistical reporting. For tab:gap and downstream stratified analyses, confidence intervals are 1000-resample task bootstraps over problems. Because each model is run once on TRACEBENCH-FULL, intervals quantify instance-level uncertainty (the dataset's problem distribution), not decoding-seed variance.

D Experimental Protocol and Cost Accounting

Model list. The six models in Table 4 are: **(1)** Qwen3.5-27B (dense, `Qwen/Qwen3.5-27B`, BF16), **(2)** Qwen3.6-27B (dense, `Qwen/Qwen3.6-27B`, BF16), **(3)** Qwen3.6-35B-A3B (MoE, 3B active, `Qwen/Qwen3.6-35B-A3B`, BF16), **(4)** GLM-4.7-Flash (MoE, `zai-org/GLM-4.7-Flash`, AWQ-int4), **(5)** DeepSeek-R1-Distill-Qwen-32B (reasoning-distilled, AWQ-int4), and **(6)** Gemini-3.1-Pro Preview (Google API, Standard tier, \$2/\$12 per million tokens).

Hardware. Local models run on a single NVIDIA H100 80 GB via vLLM with continuous batching. 32B-class models use AWQ-int4 quantization to fit comfortably alongside the KV cache; smaller models run in BF16. Gemini calls go through the Google AI Studio Standard API in synchronous mode (Batch is not used because of its

24-hour asynchronous SLA, which interacts badly with multi-turn wave dependencies).

Decoding parameters. Temperature 0.2, top-p 0.95, max output tokens 4096 per call. The maximum number of submission attempts per problem (`T_max`) is 5 for the main results; Appendix E reports sensitivity to `T_max` $\in \{3, 5, 8, 10\}$ on a single model.

Prompt. Every attempt at turn t receives the full transcript H_t, including the prior model submissions, their parsed blame spans, harness execution output, and test feedback. The prompt explicitly requests a JSON-parseable `blame_spans` field alongside the revised program. Unparseable or missing blame is counted against the model (Blame@1$=0$).

Sandbox. Each test execution runs in a fresh Python subprocess with a 60-second wall-clock timeout. RegressionRate uses a per-test wrapper (each assertion in its own `try/except`) so a single failure does not mask later results.

Task-level bootstrap. We do not run multi-seed sampling for either local or API models. Confidence intervals are 1000-resample task bootstraps over the 128 Hard problems; each resample is drawn with replacement from the fixed single-run per-problem records. This is the appropriate uncertainty quantification for a dataset paper: it captures how robust the gap is to the problem distribution, not how robust a particular sampling strategy is.

Day-0 dry-run. Before the main matrix, we run a 30-problem stratified pilot (10 Easy-Medium, 10 Hard, 10 VeryHard+) on every model to confirm prompt format, parse rates, latency, and Gemini cost projections. The pilot's parse / timeout / empty-blame counts are reported in Table 15.

Cost accounting. Table 9 in the main text reports per-model run cost. The full matrix uses approximately *11.7 H100-hours* of local compute and *$97.4* of API spend.

E Full Results and Case Studies

This appendix collects per-model split breakdowns, difficulty/depth stratification, fault-family performance, historical continuity rows, the Day-0 calibration table, an optional turn-budget sensitivity, and two complete qualitative case studies.

E.1 Failure-Mode Taxonomy Definitions

Table 11: Five-mode failure taxonomy used by the rule-based classifier in `src/evaluation/failure_modes.py`. Each mode is reconstructed from active spans, blamed spans, edit footprints, and test transitions; per-model frequencies are shown in panel (c) of Figure 4.

Mode	Trace signature	Why outcome-only misses it
`precise_repair`	Blame overlaps G_t; edits stay near G_t; tests improve.	Outcome sees success but not the causal link.
`symptom_patch`	Tests improve while blame misses G_t.	Passing can occur through exposed-test overfitting.
`semantic_drift`	Edits move into correct invariants after an early wrong hypothesis.	The turning point is hidden by the terminal label.
`regression_loop`	Previously passing tests newly fail across turns.	Terminal success may occur after wasted destabilization.
`diagnostic_recovery`	Early miss followed by later correct blame and localized repair.	A delayed run can still contain useful process evidence.

E.2 Hit-vs-Miss Stratification (numerical)

Table 12: Numerical version of the per-turn curves in Figure 4(a)-(b). We stratify trajectories on TRACEBENCH-HARD by whether the model's first blamed span overlaps the active fault, and report Miss − Hit deltas with task-bootstrap 95% CI over problems.

Downstream signal	Miss − Hit	Meaning
Cumulative patch size	*+14.2 lines* *[+10.8, +17.6]*	Early misses lead to broader later edits.
Outside-G (final)	*+19.6 pts* *[+16.1, +23.0]*	Mis-localized trajectories spread further outside G.
Final Blame@1	*−32.4 pts* *[−36.0, −28.7]*	Once the first hypothesis is wrong, attribution rarely recovers.

E.3 Difficulty-Band Breakdown

Table 13: Difficulty-band breakdown on TRACEBENCH-FULL (averaged across the six models in Table 4). Gap is roughly constant across bands; Outside-G, RegRate, and average turns rise monotonically with difficulty.

Band	n	Avg turns	Pass@1	Blame@1	Gap	Out-G
Easy / Medium	411	*2.21*	*86.3*	*46.2*	*40.1*	*14.7*
Hard	210	*3.04*	*74.5*	*33.8*	*40.7*	*21.2*
VeryHard+	190	*3.62*	*61.4*	*22.7*	*38.7*	*27.9*

E.4 Fault-Family Performance

E.5 Day-0 Calibration

E.6 Turn-Budget Sensitivity

We sweep `T_max` on a single model (Qwen3.6-35B-A3B, TRACEBENCH-HARD) to test whether the gap is an artifact of the 5-turn budget used in the main table. Pass@1 plateaus around `T_max`$=8$, but Gap *widens* monotonically with more turns: more turns gives the model more opportunities to drift outside the active fault region.

Table 14: Fault-family performance breakdown on a representative model (Qwen3.6-35B-A3B, TRACEBENCH-FULL). Dependency misuse has the largest gap, suggesting that cross-region attribution is the hardest subproblem.

Family	Count	Pass@1	Blame@1	Gap
Boundary / off-by-one	765	*87.6*	*51.8*	*35.8*
Dependency misuse	177	*72.3*	*29.4*	***42.9***
Omission / missing branch	88	*78.4*	*35.7*	*42.7*
Wrong operator / cond.	77	*81.2*	*44.6*	*36.6*
Corner-case / type	6	*66.7*	*33.3*	*33.4*

Table 15: Day-0 calibration on 30 stratified tasks per model (180 runs total). Confirms that all six models can be evaluated reliably before the Day-1 main push.

Model	n	Parse OK	Empty blame	Code OK	Timeout	Avg latency
Qwen3.5-27B	30	*28*	*3*	*29*	*1*	*18.4 s*
Qwen3.6-27B	30	*29*	*2*	*30*	*0*	*14.7 s*
Qwen3.6-35B-A3B	30	*29*	*2*	*30*	*0*	*8.3 s*
GLM-4.7-Flash	30	*27*	*4*	*29*	*1*	*11.5 s*
DeepSeek-R1-Distill-Qwen-32B	30	*28*	*2*	*29*	*1*	*24.6 s*
Gemini-3.1-Pro Preview	30	*30*	*1*	*30*	*0*	*6.2 s*

E.7 Historical Continuity Rows

We include two reference rows from prior evaluations (different model set, same protocol) to verify that the Pass–Blame gap predates the current model lineup.

E.8 Case Studies

Case 1: precise repair (`TB_HARD_00012`). A two-turn instance with an injected fault at turn 2. Across 29 turns of edit-test-revise the trajectory passes all assertions. The terminal blamed span overlaps the active fault span; Outside-G stays below 0.2 throughout; no test that passed under an earlier P_t newly fails under P_{t+1}. Classified as `precise_repair`.

Case 2: early breakdown (`TB_HARD_00000`). At turn 13 the model produces three consecutive responses that fail to include a parseable command block; the harness records the run as failed. The first blamed span never overlaps the active fault. Cumulative edit footprint grows in earlier turns; the trajectory is classified as `symptom_patch`. The interesting observation is that outcome-only evaluation collapses this into a single failure bit, while TRACEBENCH records the exact turn at which protocol drift, rather than the original fault, dominates the failure.

Case 3: diagnostic recovery (planned, third release-time case). The third case will be a trajectory where the first blamed span misses the active fault but a later turn corrects the hypothesis and converges to a localized repair (the `diagnostic_recovery` mode in the taxonomy). We will select this case from the released six-model matrix once it is collected.

Table 16: Turn-budget sensitivity (Qwen3.6-35B-A3B on TRACEBENCH-HARD). Mock values.

T_max	Pass@1	Blame@1	Gap	Avg turns used
3	*51.8*	*17.2*	*34.6*	*2.34*
5	*66.2*	*19.6*	*46.6*	*3.24*
8	*72.4*	*21.3*	*51.1*	*4.05*
10	*74.1*	*22.0*	*52.1*	*4.41*

Table 17: Historical continuity rows from prior evaluations. Numbers cached from earlier drafts; not re-run for this submission.

Model	Split	Pass@1	Blame@1	Gap
Qwen3-Coder-480B-A35B	TRACEBENCH-FULL	86.9	49.5	37.4
Qwen3-Coder-480B-A35B	TRACEBENCH-HARD	66.9	13.1	53.8 ± 5.7
Claude-4.5 Sonnet	TRACEBENCH-FULL	95.8	31.2	64.6
Claude-4.5 Sonnet	TRACEBENCH-HARD	88.8	1.7	87.1 ± 3.8

F Optional Trace-Signal Use Case

The released traces carry per-turn blame, edit footprint, and active-fault signals. Concurrent work (Anonymous Authors, 2026) uses these signals to drive an inference-time controller for traceable repair, demonstrating that the released audit channel is actionable. We mention this only to make the contribution boundary explicit: the present paper releases the dataset, the protocol, and the metrics; controller design is not the primary contribution.

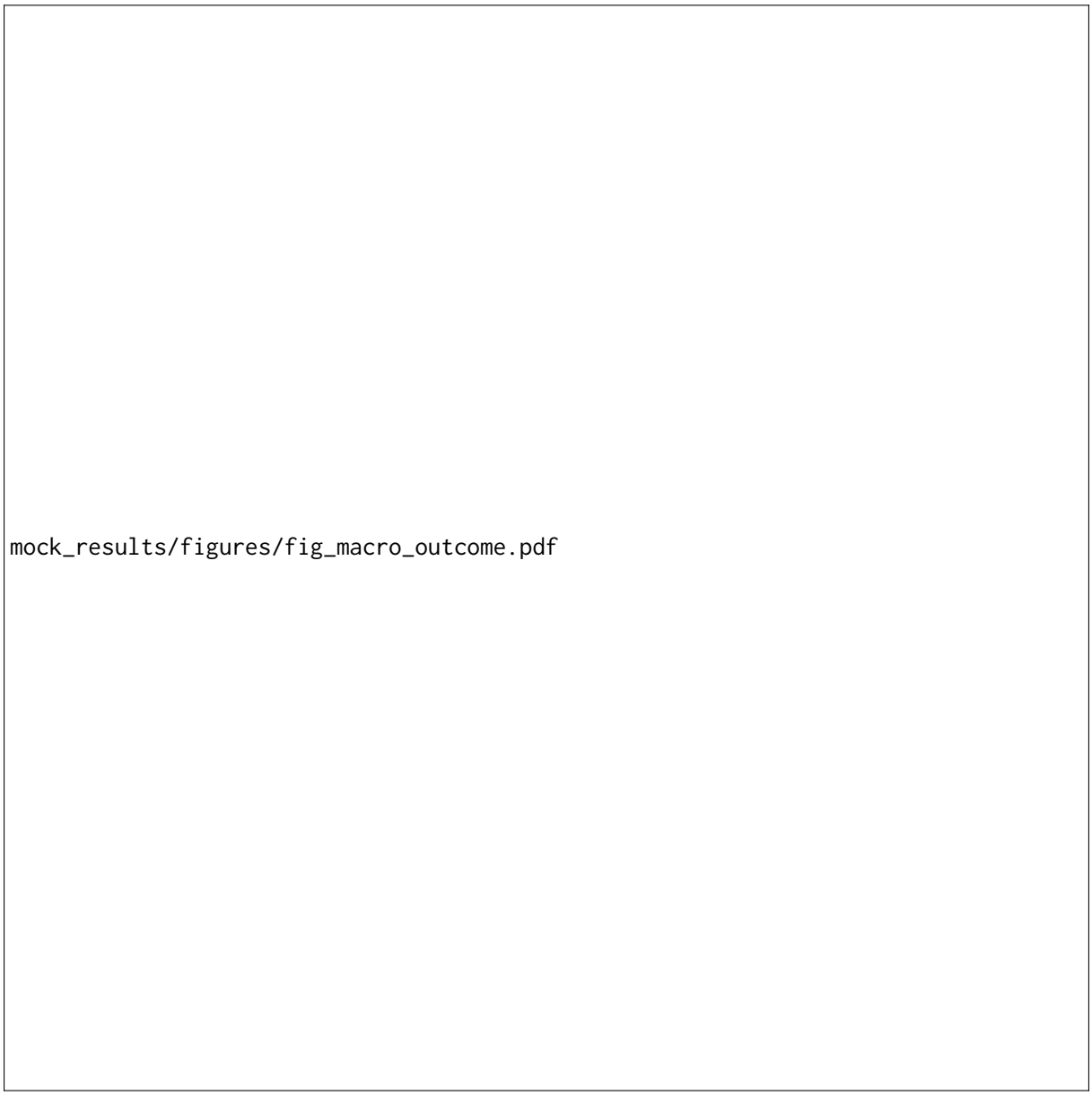

Figure 2: **Macro view: outcome and traceability diverge across models, splits, and difficulty bands.** (a) Per-model Pass−Blame gap. Every model shows a > 25 pt gap on both splits; the gap amplifies on TRACEBENCH-HARD for five of six rows. Reasoning-distilled DeepSeek-R1 has the smallest gap; closed frontier Gemini-3.1-Pro has the largest. (b) Pass vs Blame on TRACEBENCH-HARD; every marker lies below the diagonal, inside the shaded traceability-gap zone. (c) Difficulty bands on TRACEBENCH-FULL: Pass and Blame both decline with difficulty, but Gap stays roughly constant (≈ 40 pts), so TRACEBENCH-HARD amplifies process signal rather than gap magnitude.

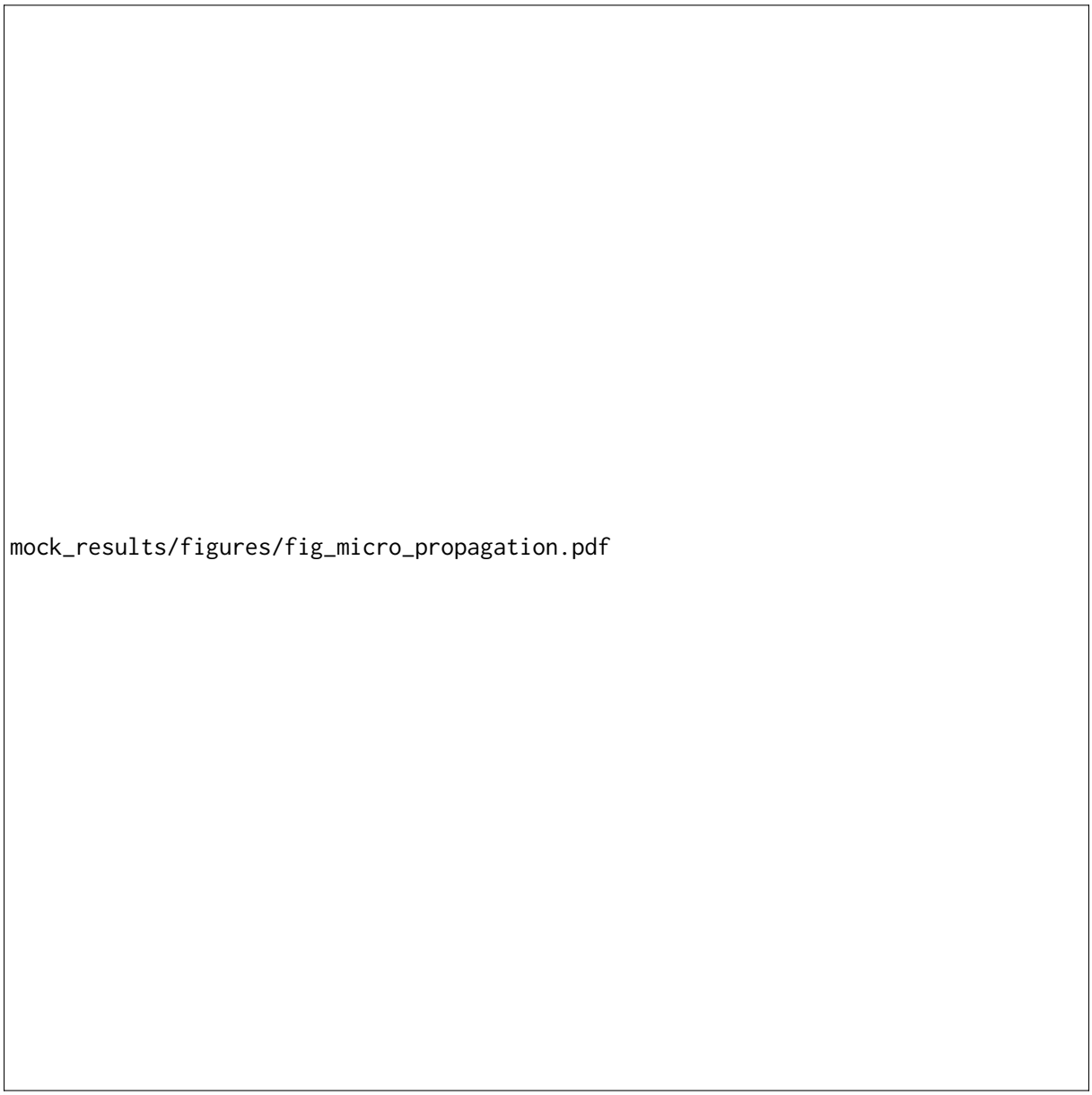

Figure 3: **Micro view: structural diffusion is behaviorally validated.** (a) Per-edit scatter of Outside-G vs. RegressionRate across all six models on TRACEBENCH-HARD, with a pooled linear fit. (b) Per-model regression slopes; all six are positive and clustered around the pooled mean, showing that the relationship is not driven by a single model family. (c) Distribution of trajectory-mean Outside-G by fault family (violins; black bar = median). Diffusion shifts upward for harder cross-region families (dependency misuse, corner-case / type) relative to local boundary errors.

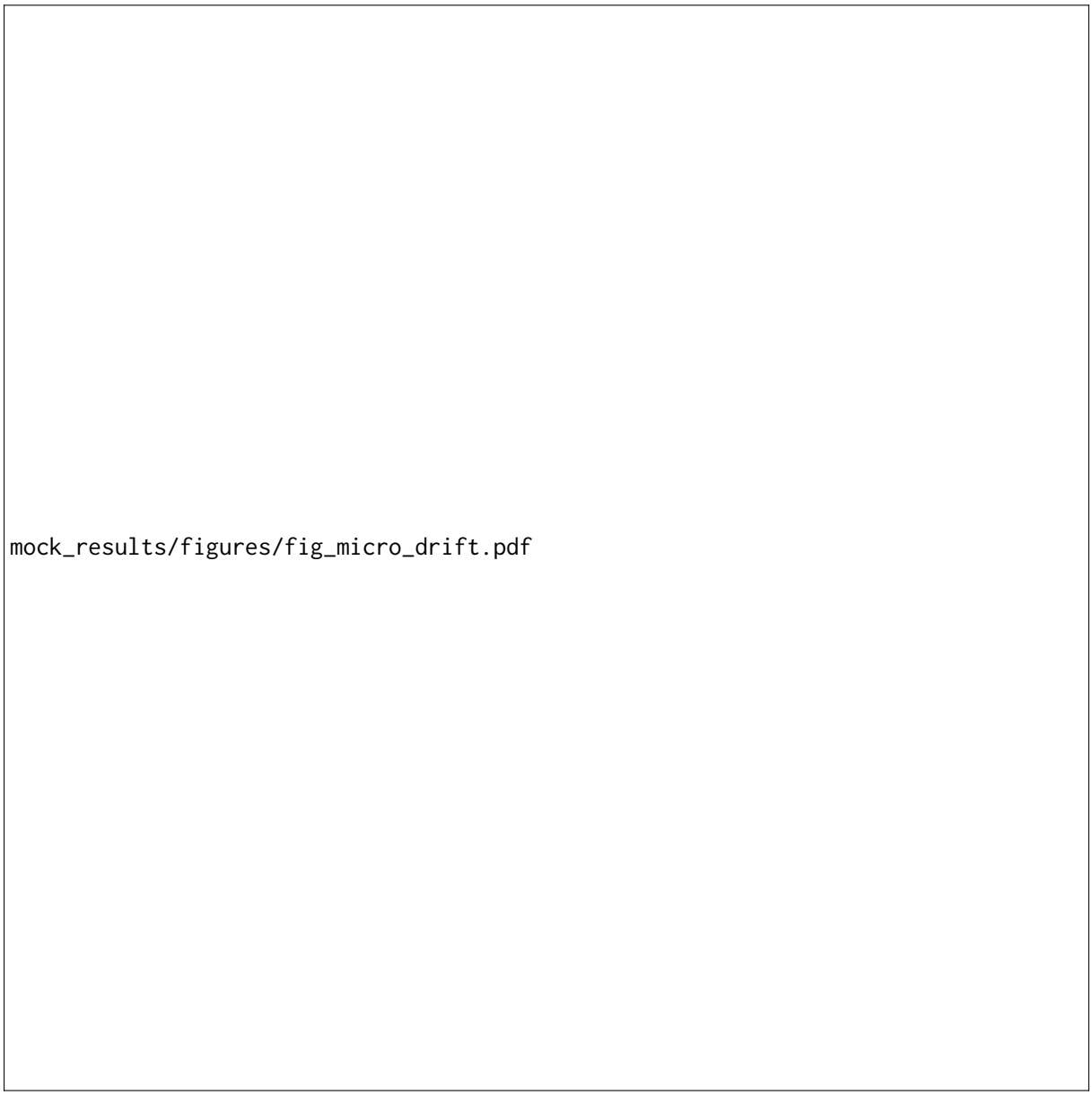

Figure 4: **Micro view: where the gap comes from.** (a) Cumulative patch size by turn, stratified by whether the model's first blamed span overlaps the active fault (Hit vs. Miss). Misses accumulate roughly twice as many edited lines by turn 5. (b) Per-turn mean Outside-G under the same stratification: Hit trajectories plateau around 0.18, Miss trajectories drift to 0.42 by turn 5. (c) Per-model failure-mode distribution on TRACEBENCH-HARD. `precise_repair` is only $\sim$1/3 of trajectories for every model; reasoning-distilled DeepSeek has the highest share (*41%*), the weakest open dense model has the lowest (*27%*).